QUILONS

SentryCode

Product & System Requirements Specification

Version 0.1

25 August 2026

Catch compliance issues at code level - before merge, before release, before audit.

Contents

1. Purpose

QUILONS SentryCode is a standalone developer and CI/CD compliance product that continuously checks software changes for security, compliance, provenance, dependency, licensing, and policy violations.

Its purpose is to catch compliance issues at code level: before merge, before release, before audit.

SentryCode SHALL be independently installable and operable while also integrating as a plugin into the wider QUILONS Compliance platform.

2. Product Positioning

SentryCode SHALL primarily serve software engineering and assurance roles and SHALL operate close to the source-code change and build lifecycle.

- Software developers
- DevOps engineers
- Platform engineers
- Security engineers
- Application security teams
- Software compliance teams

SentryCode SHALL operate primarily at:

- Developer workstation
- Git pre-commit/pre-push
- Pull request
- CI/CD pipeline
- Build
- Release gate

SentryCode SHALL NOT depend on QUILONS CRA for operation. QUILONS CRA MAY consume evidence produced by SentryCode.

3. QUILONS Compliance Integration

QUILONS Compliance SHALL act as a thin platform/UI shell. SentryCode SHALL integrate with QUILONS Compliance as an independently owned plugin.

QUILONS Installer

QUILONS Compliance

|- SentryCode

|- CRA

|- DPDP

|- EU AI Act

```
`- future compliance plugins
```

The Compliance shell SHALL:

- Discover installed plugins.
- Expose UI panes only for installed plugins.
- Obtain SentryCode capabilities through governed interfaces.
- Receive findings/evidence through defined capability APIs and/or events.

SentryCode SHALL NOT require direct access to another QUILONS module's database. Other QUILONS modules SHALL NOT access the SentryCode database directly.

Integration SHALL use:

- Versioned APIs
- Versioned capability contracts
- Governed events
- Stable evidence schemas

4. Standalone Operation

SentryCode SHALL remain usable without QUILONS CRA, the QUILONS Compliance UI, cloud connectivity, or commercial third-party security products.

Standalone usage SHALL support at minimum the following conceptual command surface. Exact command naming may evolve during CLI design.

```
sentrycode scan

sentrycode check

sentrycode policy

sentrycode sbom

sentrycode release
```

5. Core Execution Model

```
Repository / Change
    |
    v
SentryCode CLI / CI Adapter
    |
    v
Repository & Change Discovery
    |
    v
Scanner Plugins
    |
    v
```

Normalized Findings

|

v

Policy Evaluation

|

v

Decision: PASS / WARN / FAIL

|

v

Evidence + Report + Exit Code

Scanner implementations SHALL NOT directly determine the final compliance decision. Scanner results SHALL first be normalized into SentryCode findings. The policy layer SHALL determine whether those findings pass, warn, block merge, block build, or block release.

6. Finding Model

SentryCode SHALL define a stable normalized finding contract. A finding SHOULD include at minimum:

- Finding ID
- Finding type
- Scanner/source
- Rule ID
- Title
- Description
- Severity
- Repository
- Commit SHA
- Branch
- File path
- Line/range when applicable
- Package/component when applicable
- Evidence reference
- Policy result
- Timestamp
- Remediation guidance
- Suppression/waiver status

Severity SHALL support a normalized scale independent of individual scanner severity models.

7. Evidence Model

SentryCode SHALL generate machine-readable evidence suitable for SentryCode reports, QUILONS Compliance, QUILONS CRA, audit export, and future compliance modules.

Initial evidence types SHALL include:

sbom.generated

license.check

dependency.check

vuln.scan

secret.scan

policy.eval

provenance.attestation

build.attestation

commit.verification

release.gate

Evidence SHALL be:

- Immutable after issuance or revision-controlled.
- Traceable to the execution that generated it.
- Traceable to repository and commit.
- Timestamped.
- Machine-readable.
- Versioned.

Where appropriate, evidence SHOULD be cryptographically verifiable.

8. Secrets and Credential Detection

SentryCode SHALL detect likely committed credentials and secrets. Initial capability SHALL include:

- API keys
- Access tokens
- Private keys
- Certificates containing private material
- Password-like credentials
- Connection strings
- Cloud-provider credentials
- High-entropy secrets
- Configurable organisation-specific secret patterns

Detection SHALL be usable during local scan, pre-commit, pull request, CI/CD, and repository history inspection. SentryCode SHOULD prevent credentials from entering Git history whenever execution occurs before commit.

9. SBOM

SentryCode SHALL generate Software Bills of Materials. Supported output SHALL initially include at least one industry-standard format, with preference for CycloneDX or SPDX. Long-term support SHOULD include both.

SBOM generation SHALL identify:

- Packages
- Versions
- Package managers
- Dependency relationships where available
- Package hashes where available
- License information where available
- Repository/commit provenance

SentryCode SHALL support SBOM diffing between software changes. For a pull request, SentryCode SHOULD identify:

- Added dependencies
- Removed dependencies
- Upgraded dependencies
- Downgraded dependencies
- License changes
- Newly introduced vulnerable components

10. Dependency and License Governance

SentryCode SHALL evaluate software dependencies against organisational policy. Policies SHALL support:

- Allowed packages
- Denied packages
- Allowed registries
- Denied/untrusted registries
- Allowed licenses
- Denied licenses
- Review-required licenses
- Package maintenance criteria
- Version restrictions

SentryCode SHOULD detect potentially problematic licensing including configurable policies for licenses such as AGPL, GPL, LGPL, SSPL, and other organisation-defined categories.

SentryCode SHALL NOT hard-code one universal legal interpretation of software licenses. License acceptance SHALL be organisation-policy driven.

11. Vulnerability Scanning

SentryCode SHALL support Software Composition Analysis for known dependency vulnerabilities. Finding information SHOULD include:

- Vulnerability identifier
- Affected package
- Installed version
- Fixed version when available
- Severity
- Exploitability/context when available
- Source vulnerability database
- Detection timestamp

SentryCode SHALL support configurable policy thresholds, for example:

```
FAIL if CRITICAL vulnerability exists
FAIL if HIGH vulnerability has known exploit
WARN on MEDIUM
ALLOW LOW
```

12. Static Analysis / Code Security

SentryCode SHALL provide an extensible mechanism for code-level security checks.

Initial language priority:

- TypeScript / JavaScript
- Python

Future support SHOULD include:

- Java
- .NET/C#
- C
- C++
- Rust
- Go

SentryCode MAY reuse permissively licensed open-source scanners. Commercial products SHALL only be optional integrations. Mandatory SentryCode functionality SHALL NOT depend on commercial scanners.

13. Policy-as-Code

SentryCode SHALL provide policy-as-code capabilities. OPA/Rego SHOULD be supported unless implementation analysis demonstrates a stronger alternative.

Policy SHALL be able to evaluate:

- Findings
- Dependencies
- SBOM changes
- Vulnerabilities
- Secrets
- Repository properties
- Branch properties
- Commit metadata
- Build provenance
- Attestations
- Scanner outcomes

Policies SHALL support organisational inheritance:

Global organisation policy

v

Tenant policy

v

Project policy

v

Repository policy

v

Service/package policy

Lower levels SHALL only override higher policies where permitted.

14. Waivers / Exceptions

SentryCode SHALL support controlled temporary exceptions. A waiver SHALL include:

- Finding or rule
- Scope
- Reason
- Author
- Approver where required
- Created timestamp
- Expiration timestamp
- Ticket/reference where applicable

Waivers SHALL NOT silently suppress findings. Reports SHALL indicate that a finding exists but has an active waiver. Expired waivers SHALL automatically stop applying. Waiver changes SHALL be auditable.

15. Git and Repository Assurance

SentryCode SHOULD support checks including:

- Commit signature verification
- Author identity verification
- Protected branch expectations
- Required review expectations
- Commit/branch policy
- Repository provenance

Where the Git hosting provider supports it, adapters MAY verify repository governance settings. Provider-specific capability SHALL remain behind adapters and SHALL NOT leak into the core model.

16. Build Provenance

SentryCode SHALL record build provenance. Provenance SHOULD include:

- Repository
- Commit SHA
- Branch/tag
- Build ID
- Builder identity
- Build environment
- Build timestamp
- Toolchain information
- Artifact hash
- Dependency/SBOM reference

SentryCode SHOULD support signed attestations. Industry-standard attestation formats SHOULD be preferred where practical.

17. Release Gate

SentryCode SHALL provide a release gating capability. The release gate SHALL evaluate all applicable policy requirements before allowing release. Results SHALL include PASS, WARN, and FAIL.

```
sentrycode release check
```

CLI SHALL use deterministic exit codes suitable for CI/CD. Example conceptual model:

```
0 = PASS
1 = policy failure
```

2 = scan/runtime failure

3 = configuration failure

Exact exit code definitions SHALL be formally versioned.

18. CI/CD Integration

SentryCode SHALL be CI/CD-platform agnostic at its core. Adapters SHOULD support environments such as:

- GitHub Actions
- GitLab CI
- Azure DevOps
- Jenkins
- Generic shell CI

Core functionality SHALL NOT depend on any particular CI provider.

19. Monorepo Support

SentryCode SHALL support single-package repositories, multi-package repositories, monorepos, and multiple services. Policies SHALL be capable of applying independently to repository, service, package, and component.

SentryCode SHOULD avoid rescanning unaffected components where safe and deterministically possible.

20. Offline and On-Premises Operation

SentryCode SHALL support on-premises and disconnected environments. Core scanning SHALL NOT require live communication with QUILONS cloud services.

Where vulnerability intelligence requires updates, SentryCode SHALL support:

- Downloadable vulnerability database snapshots
- Controlled database synchronisation
- Organisation-hosted mirrors
- Air-gapped update procedures

21. Multi-Tenant Policy Management

When operated through QUILONS Compliance, SentryCode SHALL support tenant-isolated configuration. Tenant A SHALL NOT be able to:

- Read Tenant B policies
- Read Tenant B reports
- Read Tenant B evidence
- Apply Tenant B waivers
- Access Tenant B repositories

Tenant/project policy changes SHALL be auditable.

22. Report Formats

SentryCode SHALL produce human-readable and machine-readable output. Initial formats SHOULD include:

- Terminal output
- JSON
- SARIF where applicable
- SBOM artifact
- Evidence artifact

Future reports MAY include HTML, PDF, and Compliance dashboard summaries. The CLI SHALL remain usable without the UI.

23. Scanner Plugin Architecture

Scanner functionality SHALL be modular.

```
SentryCode Core
  |
  |- Secret Scanner
  |- SBOM Scanner
  |- License Scanner
  |- Vulnerability Scanner
  |- Static Analysis Scanner
  |- Provenance Scanner
  |- Git Assurance Scanner
  `- Future domain scanners
```

Scanner plugins SHALL communicate through versioned contracts. A scanner SHALL return normalized results rather than directly modifying global SentryCode state.

24. Automotive Compliance Extension

SentryCode SHALL be designed so automotive software checks can be introduced without modifying core architecture. Potential automotive capabilities include:

- MISRA C
- MISRA C++
- AUTOSAR C++ guidelines
- UNECE R155 evidence
- UNECE R156 evidence
- ISO/SAE 21434 engineering evidence
- Software update provenance

- Cybersecurity-relevant change evidence

These SHALL be implemented as domain-specific rules/plugins using the same finding model, policy model, evidence model, and release-gate model. Automotive rules SHALL NOT be hard-wired into generic SentryCode core.

25. QUILONS CRA Relationship

SentryCode SHALL NOT require CRA. When both products are installed:

Developer change

v

SentryCode

v

Findings + signed evidence

v

QUILONS Compliance evidence plane

v

CRA consumes relevant evidence

CRA SHALL treat SentryCode evidence as an authoritative record of the SentryCode execution that generated it. CRA SHALL NOT query SentryCode internal database tables. Without SentryCode installed, CRA SHALL continue to operate using its own available evidence and checks.

26. Security Requirements

SentryCode SHALL:

- Avoid transmitting source code externally unless explicitly configured.
- Protect secrets encountered during scanning.
- Avoid reproducing full secrets in findings or logs.
- Support redaction.
- Protect stored evidence from unauthorised modification.
- Maintain auditability for policy changes and waivers.
- Operate under least privilege.
- Avoid unnecessary repository modification.

Scanner execution SHOULD default to read-only repository access.

27. Privacy Requirements

SentryCode SHALL minimise collection of developer personal information. Developer identities used for provenance SHALL only contain information necessary for accountability, commit/build provenance, and audit evidence.

Source code SHALL not be treated as telemetry. Telemetry, if introduced, SHALL be explicitly configurable and separable from core operation.

28. Configuration

Configuration SHALL be declarative and versionable. A repository SHOULD be able to contain a SentryCode configuration such as:

```
.sentrycode/
  config.*
  policies/
  waivers/
```

Exact names/formats SHALL be decided during implementation. Organisation-managed configuration SHALL also be supported.

29. Determinism

Given the same repository state, scanner versions, vulnerability database version, policy version, and configuration, SentryCode SHOULD produce materially reproducible results.

Evidence SHALL record sufficient version metadata to explain differing results between executions.

30. Versioning

The following SHALL be versioned independently where appropriate:

- CLI
- Finding schema
- Evidence schema
- Scanner API
- Policy API
- Compliance capability API
- Report schema

Breaking contract changes SHALL require explicit schema/API version changes.

31. Auditability

SentryCode SHALL provide a traceable history of:

- Scan execution
- Policy version
- Scanner version
- Findings
- Waivers
- Policy changes
- Release decisions
- Evidence issuance

An auditor SHOULD be able to determine: what was checked, by which version, against which policy, on which source revision, and what decision resulted.

32. Performance Requirements

SentryCode SHALL support incremental execution. PR and local scans SHOULD primarily inspect changed content where doing so does not invalidate the assurance being performed.

Full repository scanning SHALL remain available. Performance optimisations SHALL NOT weaken deterministic policy enforcement.

33. Failure Behaviour

SentryCode SHALL distinguish:

- Compliance failure
- Scanner failure
- Configuration error
- Infrastructure failure
- Unsupported analysis

A scanner failure SHALL NOT silently become PASS. Policy SHALL define whether specific unavailable checks fail closed, warn, or are optional. Security-critical required checks SHOULD default to fail closed.

34. Implementation Principles

1. SentryCode remains standalone.
2. QUILONS Compliance integration is optional.
3. CRA integration is optional.
4. No cross-module database access.
5. Scanner tools are replaceable.
6. Open standards are preferred.
7. Commercial third-party products are optional integrations only.
8. Core architecture remains domain-neutral.
9. Automotive capabilities are plugins/rule packs.
10. Findings and evidence are stable first-class contracts.
11. Policy determines enforcement.
12. CLI/CI operation remains first-class even when a UI exists.

35. Initial Implementation Baseline

The first implementation milestone SHOULD establish the architecture rather than attempting every scanner. The baseline SHALL implement:

```
CLI
 |
```

```
|- Repository context
|- Configuration loader
|- Scanner plugin interface
|- Finding contract
|- Evidence contract
|- Policy execution boundary
|- Report generation
`- Stable exit codes
```

At least one real scanner should exercise the architecture end-to-end. Recommended first scanner: Secrets detection.

Recommended subsequent sequence:

13. SBOM
14. Dependency/license policy
15. Vulnerability scanning
16. Policy-as-code
17. Provenance/attestation
18. Release gate
19. CI adapters
20. Compliance integration
21. Domain-specific automotive rules

36. Definition of Product Boundary

SentryCode answers:

> ***"Is this software change/build/release compliant with the engineering policies we require, and can we prove what was checked?"***

QUILONS CRA answers a broader assurance question:

> ***"Can the organisation demonstrate that the product/process satisfies the applicable Cyber Resilience Act requirements?"***

QUILONS Compliance provides the common product surface through which these and future compliance plugins can be discovered and used.

37. Non-Goals

SentryCode SHALL NOT initially attempt to become:

- A complete SIEM
- An EDR product
- A penetration-testing platform
- A runtime network security platform
- A generic issue tracker

- A source-code hosting platform
- A replacement for Git
- A replacement for CI/CD
- A general GRC platform

Its primary domain is continuous software engineering compliance and evidence at code/build/release level.

Document status: Requirements baseline v0.1